Preliminary Concepts for Data Structures

Dr. Nafis Sadeq
Assistant Professor
Department of Computer Science and Engineering
East West University



Outline

1. Data Types
2. Pointers
3. Structures
4. Dynamic Memory
5. ADTs
6. Asymptotic Notations
7. Complexity Analysis Examples

Data Types

C++ provides several fundamental types used throughout data structures:

Category	Examples	Notes
Integer	`short`, `int`, `long`, `long long`	Signed/unsigned variants; size can vary by platform. Useful for indexing, counts.
Floating-point	`float`, `double`, `long double`	Real numbers (rounding error). Use `double` for numerical stability.
Character	`char`, `wchar_t`, `char16_t`, `char32_t`	Stores code units; `char` often 1 byte.
Boolean	`bool`	`true`/`false`. Often 1 byte storage.

Tip: Use fixed-width types (e.g., `int32_t`, `uint64_t`) for predictable sizes.

Data Types

```cpp
#include <iostream>
using namespace std;

int main() {
  int    a = 42;          // integer
  double b = 3.14159;     // floating point
  char   c = 'X';         // character

  // Sizes may vary by platform
  cout << "sizeof(int) = "    << sizeof(int)    << "\n";
  cout << "sizeof(double) = " << sizeof(double) << "\n";
  cout << "sizeof(char) = "   << sizeof(char)   << "\n";
}
```

Common Pitfalls with Numeric Types

- **Overflow/Underflow:** e.g., adding beyond `INT_MAX` wraps (undefined behavior in signed overflow).
- **Integer Division:** `5/2 == 2`. Cast to `double` when needed: `5/2.0`.
- **Precision:** `float` has 7 digits; prefer `double` for most DS work.
- **Signed vs Unsigned:** Mixing can cause surprising comparisons/underflow.

What is a Pointer?

- A **pointer** stores the *memory address* of another object.
- Crucial for implementing dynamic data structures (linked lists, trees, graphs).
- Operators: `&` (address-of), `*` (indirection/dereference).

Analogy: Think of a pointer as a "map location" to a house (the object). The map is not the house, but lets you get there.

Basic Pointer Usage

```cpp
#include <iostream>
using namespace std;

int main() {
  int value = 10;       // a normal integer variable
  int* p = &value;      // p holds the address of value

  cout << "value = " << value << "\n";
  cout << "&value = " << &value << "\n";
  cout << "p (address) = " << p << "\n";
  cout << "*p (dereference) = " << *p << "\n";

  *p = 25; // write via pointer
  cout << "value (after *p=25) = " << value << "\n";
}
```

What is a Structure (`struct`)?

- A `struct` groups related fields into one composite type.
- Useful for bundling data together, e.g., nodes in linked lists, points in geometry, or student records.
- In C++, `struct` members are public by default.
- Provides a foundation for understanding more advanced constructs like `class`.

What is a Class (`class`)?

- A `class` is a blueprint for creating objects (like a more powerful struct).
- Groups data (attributes) and functions (methods) together.
- In C++, members are **private by default**.
- Supports object-oriented programming principles: encapsulation, abstraction, inheritance, polymorphism.

Demo: Defining and Using a class

```cpp
#include <iostream>
using namespace std;

class Student {
public:
  int id;
  string name;
  double cgpa;
  Student(int i, string n, double c){
    id = i;
    name = n;
    cgpa = c;
  }
};

int main() {
  Student s1(201, "Sara", 3.88);
}
```

Dynamic Memory Allocation

- Allocate at runtime using `new` / `new[]` and release with `delete` / `delete[]`.
- Enables flexible, variable-size structures: linked lists, trees, hash tables.
- **Lifetime** is controlled manually; forgetting `delete` leaks memory.
- Safer alternative: RAII smart pointers (e.g., `std::unique_ptr`).

Demo: new/delete for Scalars

```cpp
#include <iostream>
using namespace std;

int main() {
  int* p = new int(42);
  cout << "*p = " << *p << " at " << p << "\n";

  delete p;
}
```

Demo: Dynamic Arrays with new[]/delete[]

```cpp
#include <iostream>
using namespace std;

int main() {
  int n = 5;
  int* arr = new int[n];

  for (int i = 0; i < n; i++){
    arr[i] = i * i;
  }
  for (int i = 0; i < n; i++){
    cout << arr[i] << ' ';
  }
  cout << "\n";

  delete[] arr;
}
```

What is an Abstract Data Type (ADT)?

- An **Abstract Data Type (ADT)** is a logical description of how data is organized and which operations are allowed on it.
- Focuses on *what operations* can be performed, not on *how* they are implemented.
- Separates **specification** (interface) from **implementation** (details hidden).
- Examples: Stack, Queue, List, Deque, Map, Set.

Different Types of ADTs

Common ADTs studied in data structures:

- **List** – Ordered collection of elements (e.g., ArrayList, Linked List).
- **Stack** – LIFO (Last In, First Out). Operations: push, pop, top.
- **Queue** – FIFO (First In, First Out). Operations: enqueue, dequeue.
- **Deque** – Double-ended queue, supports insertion/deletion at both ends.
- **Priority Queue / Heap** – Elements associated with priority, supports extraction of highest/lowest priority.
- **Map / Dictionary** – Collection of key–value pairs (e.g., HashMap, TreeMap).
- **Set** – Collection of unique elements.

Introduction to Asymptotic Notations

- Asymptotic notations describe the **growth rate** of an algorithm's time or space complexity.
- They provide a high-level understanding of performance as input size $n \to \infty$.
- Help compare algorithms independently of machine, compiler, or implementation details.
- Focus is on the most significant terms, ignoring constants and lower-order terms.

Asymptotic Bounds

- **Big-O (O):** Asymptotic *upper bound*.
  - Worst-case growth rate.
  - Example: Linear search has $O(n)$ time complexity.
- **Big-Omega (Ω):** Asymptotic *lower bound*.
  - Best-case growth rate.
  - Example: Linear search has $\Omega(1)$ (if found at first index).
- **Big-Theta (Θ):** Asymptotic *tight bound*.
  - Matches both upper and lower bounds.
  - Example: Linear search has $\Theta(n)$.

Linear Search: Complexity Analysis

- Searches sequentially for a target in an array of size n.
- **Time Complexity:**
  - Best case: $\Omega(1)$ (target at first position).
  - Worst case: $O(n)$ (target absent or at end).
  - Tight bound: $\Theta(n)$.
- **Space Complexity:** $O(1)$ (only a few variables).

Linear Search: C++ Implementation

```cpp
#include <iostream>
using namespace std;

int linearSearch(int arr[], int n, int key) {
  for (int i = 0; i < n; i++) {
    if (arr[i] == key) return i;  // found
  }
  return -1;  // not found
}

int main() {
  int arr[] = {10, 20, 30, 40, 50};
  int n = 5, key = 30;
  int result = linearSearch(arr, n, key);
  if (result != -1) cout << "Found at index " << result;
  else cout << "Not found";
}
```

Binary Search: Complexity Analysis

- Works on sorted arrays by repeatedly halving the search space.
- **Time Complexity:**
  - Best case: $\Omega(1)$ (target at mid).
  - Worst case: $O(\log n)$ (halves until size 1).
  - Tight bound: $\Theta(\log n)$.
- **Space Complexity:** $O(1)$ (iterative) or $O(\log n)$ (recursive stack).

Binary Search: C++ Implementation

```cpp
#include <iostream>
using namespace std;

int binarySearch(int arr[], int n, int key) {
  int low = 0, high = n - 1;
  while (low <= high) {
    int mid = (low + high) / 2;
    if (arr[mid] == key) return mid;
    else if (arr[mid] < key) low = mid + 1;
    else high = mid - 1;
  }
  return -1;  // not found
}

int main() {
  int arr[] = {10, 20, 30, 40, 50};
  int n = 5, key = 40;
  int result = binarySearch(arr, n, key);
  if (result != -1) cout << "Found at index " << result;
  else cout << "Not found";
}
```

Binary Search on a Sorted Array (Trace)

$A = \langle 2, 5, 8, 12, 15, 18, 21, 25, 29, 33, 37, 41, 46, 50, 54, 59, 63, 68, 72, 77, 81 \rangle, \quad \text{key} = 50$

Step	**low**	**mid**	**high**	$A[\textbf{mid}]$	**Action**
1	0	10	20	37	$37 < 50 \Rightarrow$ set $low \leftarrow 11$
2	11	15	20	54	$54 > 50 \Rightarrow$ set $high \leftarrow 14$
3	11	12	14	46	$46 < 50 \Rightarrow$ set $low \leftarrow 13$
4	13	13	14	50	$50 == 50 \Rightarrow$ **found at index 13**

Insertion Sort: Complexity Analysis

- Sorts by building a sorted portion of the array one element at a time.
- **Time Complexity:**
  - Best case: $\Omega(n)$ (already sorted).
  - Worst case: $O(n^2)$ (reverse sorted).
  - Tight bound: $\Theta(n^2)$.
- **Space Complexity:** $O(1)$ (in-place sorting).

Insertion Sort: C++ Implementation

```cpp
#include <iostream>
using namespace std;

void insertionSort(int arr[], int n) {
  for (int i = 1; i < n; i++) {
    int key = arr[i];
    int j = i - 1;
    while (j >= 0 && arr[j] > key) {
      arr[j + 1] = arr[j];
      j--;
    }
    arr[j + 1] = key;
  }
}

int main() {
  int arr[] = {5, 2, 4, 6, 1, 3};
  int n = 6;
  insertionSort(arr, n);
  for (int i = 0; i < n; i++) cout << arr[i] << " ";
}
```

Insertion Sort on an Array

$$A = \langle 5, 2, 4, 6, 1, 3 \rangle$$

Pass	Step	key	Action	Array
1	1	2	compare 2 with 5 $\Rightarrow$ swap	$\langle \mathbf{2}, 5, 4, 6, 1, 3 \rangle$
2	1	4	compare 4 with 5 $\Rightarrow$ swap	$\langle 2, \mathbf{4}, 5, 6, 1, 3 \rangle$
2	2	4	compare 4 with 2 $\Rightarrow 4 > 2$ stop	$\langle 2, \mathbf{4}, 5, 6, 1, 3 \rangle$
3	1	6	compare 6 with 5 $\Rightarrow 6 > 5$ stop	$\langle 2, 4, 5, \mathbf{6}, 1, 3 \rangle$
4	1	1	compare 1 with 6 $\Rightarrow$ swap	$\langle 2, 4, 5, \mathbf{1}, 6, 3 \rangle$
4	2	1	compare 1 with 5 $\Rightarrow$ swap	$\langle 2, 4, \mathbf{1}, 5, 6, 3 \rangle$
4	3	1	compare 1 with 4 $\Rightarrow$ swap	$\langle 2, \mathbf{1}, 4, 5, 6, 3 \rangle$
4	4	1	compare 1 with 2 $\Rightarrow$ swap	$\langle \mathbf{1}, 2, 4, 5, 6, 3 \rangle$

Insertion Sort on an Array

$$A = \langle 5, 2, 4, 6, 1, 3 \rangle$$

Pass	Step	key	Action	Array
5	1	3	compare 3 with 6 $\Rightarrow$ swap	$\langle 1, 2, 4, 5, \mathbf{3}, 6 \rangle$
5	2	3	compare 3 with 5 $\Rightarrow$ swap	$\langle 1, 2, 4, \mathbf{3}, 5, 6 \rangle$
5	3	3	compare 3 with 4 $\Rightarrow$ swap	$\langle 1, 2, \mathbf{3}, 4, 5, 6 \rangle$
5	4	3	compare 3 with 2 $\Rightarrow 3 > 2$ stop	$\langle 1, 2, \mathbf{3}, 4, 5, 6 \rangle$